Motion-Triggered Security Camera Using OpenCV

CSC 126 (BJY21) - Computer Vision | Midterm Project

Members: Andrea Glecy Semine; Beverly Nicole Siton; Jecil Mae Vesagas

1. Project Title and Objectives

Title: Motion-Triggered Security Camera Using Frame Differencing in OpenCV.

General objective: To design and implement a working computer vision application in Python and OpenCV that monitors a webcam feed (or a recorded video), detects movement in real time, and automatically saves evidence of the detected motion.

Specific objectives:

- Apply image operations (resizing, grayscale conversion, Gaussian blur) to prepare each frame for analysis.
- Segment moving regions from the static background using frame differencing, thresholding, and dilation.
- Detect and track moving objects using contours and bounding boxes, and count them on every frame.
- Display a clear on-screen status (MONITORING / MOTION DETECTED), object count, and date and time.
- Automatically save timestamped screenshots when motion occurs, with a cooldown to prevent flooding the storage.

2. Description of the Problem / Use Case

Small homes, shops, offices, and school rooms often need basic surveillance, but commercial security systems can be expensive, and manually watching camera footage for long hours is tiring and unreliable. Recording continuously also wastes storage because most footage contains nothing happening. This project addresses the problem with a low-cost solution that uses only an ordinary webcam and free software. The system stays idle in a MONITORING state and reacts only when something moves: it highlights the moving object, raises a visible alert, and stores a screenshot with the date and time, so a person only needs to review the saved captures instead of hours of video.

3. Techniques Used

The project uses three of the laboratory topics: **(a) Image Operations**, **(b) Image Segmentation**, and **(e) Motion and Object Tracking**. The table below summarizes the OpenCV functions applied and the reason for each.

Topic	OpenCV function	Purpose in this project
a. Image Operations	`cv2.resize`	Scales every frame to 800x600 so processing speed and thresholds are consistent regardless of camera resolution.
a. Image Operations	`cv2.cvtColor` (BGR to gray)	Removes color information. Motion is detected from brightness changes, so one channel is enough and it is faster to process.
a. Image Operations	`cv2.GaussianBlur` (21x21)	Smooths the image to suppress camera sensor noise and tiny lighting flicker that would otherwise appear as false motion.
b. Image Segmentation	`cv2.absdiff`	Computes the absolute pixel difference between the previous and current frame. Static areas become near-black; moving areas become bright.
b. Image Segmentation	`cv2.threshold` (25, binary)	Converts the difference image into a binary motion mask: pixels that changed by more than 25 intensity levels become white (255), the rest black (0).
b. Image Segmentation	`cv2.dilate` (2 iterations)	Expands the white regions so that fragmented parts of one moving object merge into a single solid blob.

Topic	OpenCV function	Purpose in this project
e. Motion and Object Tracking	`cv2.findContours` (RETR_EXTERNAL)	Extracts the outer boundary of every white blob in the mask. Each contour is a candidate moving object.
e. Motion and Object Tracking	`cv2.contourArea`	Filters out small blobs below MIN_CONTOUR_AREA (3000 pixels) so only meaningful movement is counted.
e. Motion and Object Tracking	`cv2.boundingRect`, `cv2.rectangle`	Finds and draws a green bounding box around each moving object, which also provides the per-frame moving object count.
Output / Visualization	`cv2.putText`, `cv2.imshow`, `cv2.imwrite`	Draws the status, object count, and date/time overlays; shows the live feed and the motion mask; saves timestamped screenshots.

Processing pipeline. For every frame the program (1) reads and resizes the frame, (2) converts it to grayscale and blurs it, (3) computes the absolute difference with the previous grayscale frame, (4) thresholds and dilates the difference to build the motion mask, (5) finds contours and discards small ones, (6) draws bounding boxes, the status text, the moving object count, and the date and time, (7) saves a screenshot if motion is present and the 5-second cooldown has passed, and (8) displays the result and the motion mask. The current grayscale frame then becomes the previous frame for the next iteration. The program exits when the user presses Q.

4. Screenshots of Results

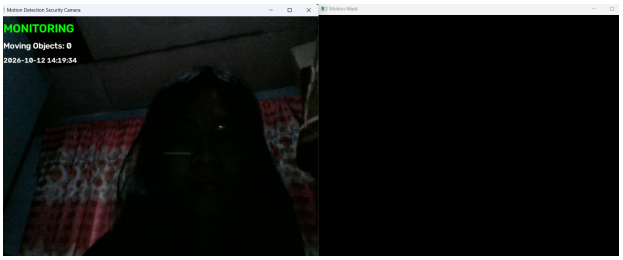


Figure 1. Normal monitoring state (green MONITORING label, count of 0).

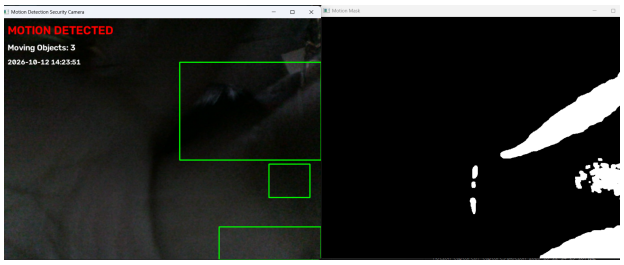


Figure 2. Motion detected: red MOTION DETECTED label, bounding box, object count, date and time.

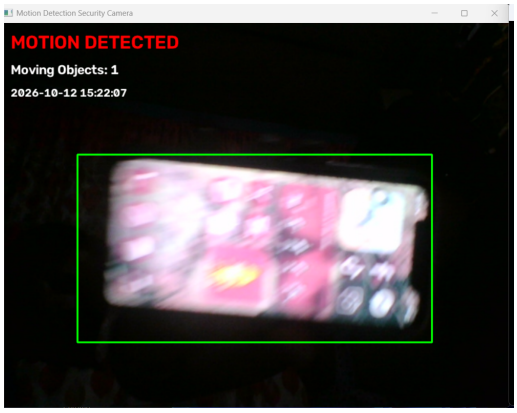


Figure 3. Close-up of the green bounding box drawn around a moving object.

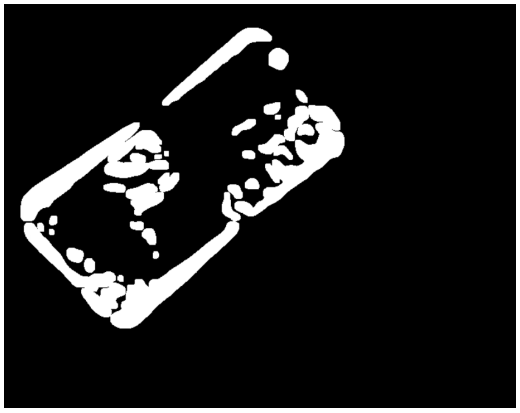


Figure 4. Motion Mask window: white regions are pixels that changed between frames.

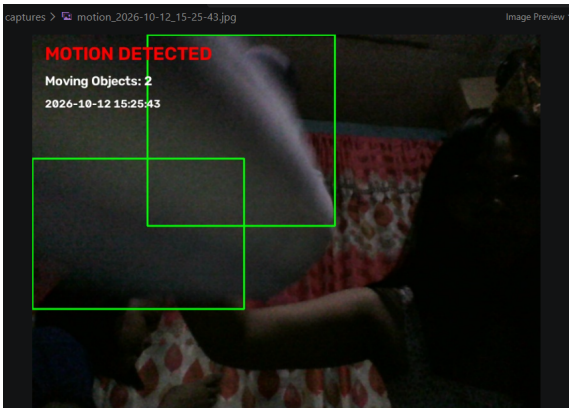


Figure 5. Automatically saved screenshot in the captures folder (timestamped filename).

5. Testing, Accuracy, Behavior, and Limitations

Behavior. When the scene is still, the difference image is almost entirely black, the motion mask is empty, and the display shows the green MONITORING label with a count of 0. When a person or object moves, the changed pixels form white regions in the mask, a green bounding box appears around each region larger than the minimum area, the label changes to a red MOTION DETECTED, and the moving object count updates every frame. A screenshot is saved at most once every 5 seconds, so a long movement produces a small number of useful captures instead of hundreds of nearly identical files.

Testing and tuning. To see how the settings affect detection, three configurations were run on the same recorded video (272 frames): Test A (Sensitive), Test B (Original, the program's default), and Test C (Less Sensitive). For each run, the program counted the frames in which motion was flagged, the approximate motion events, and the bounding boxes drawn.

Test	Min area	Threshold	Dilation	Frames with motion	Detection rate	Motion events (approx.)	Boxes drawn
A - Sensitive	500	20	2	231 / 272	84.93%	44	2106
B - Original	3000	25	2	187 / 272	68.75%	50	418
C - Less Sensitive	5000	30	3	117 / 272	43.01%	41	164

Table 1. Results of the three settings on the same 272-frame video.

Observation	Test A - Sensitive	Test B - Original	Test C - Less Sensitive
Small movement	Detected	Detected	Not detected
Large movement	Detected	Detected	Detected
Slow movement	Detected	Detected	Detected
False detections	Few	Few	Few
Bounding box accuracy	Good	Good	Good

Table 2. Observations from reviewing the output of each test.

Accuracy. Lowering the threshold and the minimum contour area made the system more sensitive: motion was flagged in 84.93% of frames in Test A, 68.75% in Test B, and 43.01% in Test C. Tests A and B detected small movements, while Test C missed them because its larger minimum area (5000) and higher threshold (30) filter them out. All three tests detected large and slow movements, with few false detections and good bounding box placement. The detection rate is the share of frames where motion was flagged; it measures sensitivity, not correctness against a ground truth, so a higher rate does not automatically mean a better result. Test A also drew 2106 boxes over 231 motion frames (about 9 per frame) compared with about 2 per frame in Test B, which likely means its very small area limit splits one movement into many small boxes. For this reason, **Test B was kept as the program's default**: it still detects small movements while giving cleaner boxes and a more readable object count. **Test A is recommended when detecting small movement is the main priority**,

provided the output is checked for false detections. The object count is the number of motion regions, not an exact count of individual people or objects.

Limitations.

- **Lighting changes:** sudden brightness changes, camera auto-exposure adjustment, or flickering lights can be detected as motion.
- **Shadows:** moving shadows change pixel brightness and may be detected as motion.
- **Stationary or very slow objects:** because only two consecutive frames are compared, an object that stops moving, or moves very slowly, produces little difference and disappears from detection.
- **Merged or split detections:** two nearby moving objects can merge into one blob, and with a very small minimum area one object can split into many boxes, so the count is approximate.
- **No identification:** the system detects that something moved, but cannot tell whether it is a person, an animal, or a curtain moved by wind.
- **Camera movement:** if the camera itself shakes or moves, the whole frame changes and everything appears as motion.
- **Limited testing:** all three tests used a single video. Other lighting conditions, scenes, and movement speeds still need to be tested.

6. Conclusion

The project successfully combined image operations, image segmentation, and motion tracking to build a functional motion-triggered security camera using only classical OpenCV techniques. It processes a live webcam feed (or a recorded video), visualizes its results with bounding boxes, labels, overlays, and a motion mask, and automatically stores timestamped evidence. Through this work, the team learned how preprocessing strongly affects detection quality (blurring removes noise, and thresholding and dilation shape the final mask), how parameter tuning balances sensitivity against false alarms (the more sensitive Test A flagged motion in 84.93% of frames, compared with 43.01% for the less sensitive Test C), and how simple methods can solve a practical problem without machine learning.

Possible improvements: replace frame differencing with background subtraction (`cv2.createBackgroundSubtractorMOG2`) to handle slow movement and lighting changes better; add a region-of-interest zone so only important areas trigger alerts; add sound, email, or SMS notifications; record short video clips instead of single images; and combine the system with a Haar cascade or other detector to confirm that the moving object is a person.